

3 Laboratorinis darbas. SQL Injekcijos

SQL Injekcija

SQL Injekcija (SQLi) - tai kodo injekcijos metodas, kuris leidžia vykdyti iš anksto nenumatytas užklausas duomenų bazėje. Užpuolikai gali pasinaudoti šiuo pažeidžiamumu, jei duombazėje yra dinaminių užklausių, naudojančių *string* sujungimą ir vartotojo pateiktą įvestį.

Pavojai

Pasinaudojęs SQL injekcijomis, užpuolikas duombazėje gali pridaryti žalos:

- Apeiti autentifikacijos mechanizmus
- Perskaityti duomenis
- Pakeisti ar ištrinti duomenis
- Sutrikdyti duombazės darbą

Pažeidžiamumas

Tarkime, turime funkciją login, kuri priima du parametrus username ir password.

```
def login(username, password):  
    query = "select name, email from users " +  
            "where username = '" + username +  
            "' and password = '" + password + "';"  
    return execute(query)
```

Kai vartotojas, kurio username = "vardas", o password = "slaptazodis" bandys prisijungti, duombazėje bus įvykdyta ši užklausa

```
select name, email from users  
where username = 'vardas' and password = 'slaptazodis';
```

Vietoj username ar password reikšmių vartotojas gali įvesti tokią reikšmę, kuri pakeis SQL užklauskos prasmę.

Autorizacijos apėjimas

Jei vartotojas įves `username = '' or 1 = 1; --` ir `password = "2"`, bus įvykdyta užklausa

```
select name, email from users
where username = '' or 1 = 1; -- and password = '2';
```

Duomenų perskaitymas

Kai `username = '' union select username, password from users; --`

```
select name, email from users
where username = ''
union
select username, password from users; -- and password = '2';
```

Duomenų pakeitimas

Kai `username = ''; update users set password = '1'; --`

```
select name, email from users
where username = '';
update users set password = '1'; -- and password = '2';
```

Duomenų ištrynimas

Kai username = ''; drop table users; -- "

```
select name, email from users
where username = '';
drop table users; -- and password = '2';
```

DDoS ataka

```
select name, email from users
where username = '';
select tab1 from (select decode(encode(convert(compress(post) using latin1),concat(post,
    post,post,post)),sha1(concat(post,post,post,post))) as tab1 from table_1);
-- and password = '2';
```

SQL injekcijas būdai

Užpuolikas šālingā kodā j aplikacijā gali ivesti per:

- aplikācijas formas laukus
- URL parametrus
- aplikācijas *cookies*
- API uzklausas
- ...

Prevencijas būdai

Apsisāugoti nuo SQL injekciju yra du būdai:

- Nenaudoti dinaminių užklausių su *string* apjungimu
- Neleisti kenkėjiškai įvesčiai būti įtrauktai į užklausas

Prepared statements

Saugiausias būdas rašyti dinamines duombazės užklaudas yra naudoti iš anksto paruoštas užklaudas (angl. *prepared statements*) su nurodomais parametrais. Naudojant šį užklausių rašymo stilių, duomenų bazė visada skirs kodą ir duomenis, neatsižvelgiant į tai, kokia vartotojo įvestis pateikiama. Be to, *prepared statements* užtikrina, kad užpuolikas negalės pakeisti užklaudos prasmės, net jei ir bus įterptos SQL komandos.

Oracle .NET

```
var query = "select name, email from users " +  
    "where username = :username and password = :password;";  
  
var command = new OracleCommand(query, connection);  
command.Parameters.Add(new OracleParameter("username", username));  
command.Parameters.Add(new OracleParameter("password", password));  
  
return command.ExecuteReader();
```


Stored procedure

Duomenų bazės procedūros (angl. *stored procedure*) - suteikia tokį pat saugumą kaip ir *prepared statements*, jei procedūros yra implementuojamos saugiai. Skirtumas tarp *prepared statements* ir procedūrų yra tas, kad procedūros SQL kodas yra aprašytas ir saugomas pačioje duomenų bazėje, o tada išskviečiamas iš aplikacijos.

Oracle .NET

```
var command = connection.CreateCommand();

command.CommandText = "myschema.login";
command.CommandType = CommandType.StoredProcedure;
command.Parameters.Add(new OracleParameter("username", OracleDbType.Varchar2, username,
    ParameterDirection.Input));
command.Parameters.Add(new OracleParameter("password", OracleDbType.Varchar2, username,
    ParameterDirection.Input));
command.Parameters.Add(new OracleParameter("name", OracleDbType.Varchar2,
    ParameterDirection.Output));
command.Parameters.Add(new OracleParameter("email", OracleDbType.Varchar2,
    ParameterDirection.Output));

return command.ExecuteReader();
```

Iš anksto nurodytos reikšmės

Kai kurios duomenų bazės užklausos negali naudoti anksčiau nurodytų parametrizacijos būdų. Pvz, lentelių ar stulpelių pavadinimai, rikiavimo kryptis (ASC, DESC). Tokios užklausos dažniausiai nurodo prastą duomenų bazės dizainą, tačiau jos gali būti saugios, jei užklausų parametrai bus nustatomi kode, o ne iš vartotojo įvesties.

.NET

Tarkime, turime užklausą, kur pagal vartotojo įvestį duomenys traukiami iš skirtingų lentelių. Vartotojas gali pasirinkti lentelę nurodydamas parametą userInput.

```
string tableName;  
switch(userInput):  
    case "Value1":  
        tableName = "fooTable";  
        break;  
    case "Value2":  
        tableName = "barTable";  
        break;  
    ...  
    default:  
        throw new InputValidationException("unexpected value provided for table name");  
  
var query = "select * from " + tableName + ";
```

Vartotojo įvesties sanitizacija

Yra du būdai kaip galima sanitizuoti vartotojo įvestį:

- 1 Leisti vartotojui įvesti tik tam tikrus simbolius. Jei vartotojo įvestis būtų sudaryta tik iš saugių simbolių (pvz. abėcėlės raidžių ir skaičių), vartotojas negalės sudaryti kenkėjiškų įvesčių.
- 2 (Nerekomenduojama) Apdoroti (angl. *escape*) simbolius. SQL injekcijos dažniausiai įvyksta dėl netinkamai apdrotų kabučių vartotojo įvestyje. Jas apdoroti galima jas pavertus į teksto simbolį. Pvz. ' → \'. Šis metodas negali garantuoti, kad bus išvengta visų SQL injekcijų.

Least privilege

Kad būtų sumažinta sėkmingos SQL injekcijos atakos žala, aplikacijos bendravimui su duombaze turėtų būti naudojami *db_vartotojai* turintys minimalią reikalingą prieigos teisę.

- Atvejams, kai reikalinga tik skaitymo teisė iš aplikacijos į DB neturėtų būti jungiamasi su administratoriaus teisės turinčiu *db_vartotoju*.
- Kiekvienai aplikacijai turėtų būti sukurtas atskiras *db_vartotojas*. Saugiau teises pridėti, nei atimti.
- Kai *db_vartotojui* reikalinga pasiekti tik dalį lentelės duomenų, turėtų būti naudojami *view*.
- Duomenų bazės veikimą implementavus procedūromis, *db_vartotojui* nebereikia suteikti *SELECT* teisių į lenteles. Užtenka suteikti *EXECUTE* teises į reikalingas procedūras.

Iš anksto nurodytos reikšmės

Prieš vykdant užklausą su vartotojo parametrais, turėtų būti patikrinta ar vartotojas tikrai turi teisę atlikti užklausą su nurodytais parametrais.

Pavyzdys

Tarkime, turime aplikaciją, kurioje vartotojas gali prisijungti savo vartotojo vardu ir slaptažodžiu. Šioje aplikacijoje yra funkcija leidžianti peržiūrėti vartotojo duomenis (vardą, pavardę, el. paštą, adresą...). Kad šie duomenys būtų parodyti, vartotojas išsiunčia užklausą į serverį su savo vartotojo id. Jei vartotojas užklausoje nurodys ne savo id, ir aplikacija netikrins ar vartotojas gali peržiūrėti su tuo id susijusius duomenis, vartotojui bus atrinkti ir parodyti kito vartotojo duomenys.

Didžiausios atakos naudojusios SQL injekcijas

- 2008, iš *Heartland Payment Systems*, *7-Eleven*, *Hannaford Brothers* ir kitų buvo pavogta 130 mln. kreditinių kortelių duomenų. Ataką surengė amerikietis Albert Gonzalez ir du nežinomi rusai. Užpuolikai ketino duomenis parduoti. Gonzalez buvo nuteistas 20 metų kalėjimo. Heartland, kaip kompensaciją savo vartotojams išmokėjo 200 mln. dolerių.
- 2011 ne pelno siekianti grupė *LulzSec* iš *Sony Pictures* pavogė ir viešai paskelbė 1 mln. vartotojų prisijungimo duomenų, 3.5 mln. skaitmeninių kuponų, ir kitų duomenų. Slaptažodžiai buvo saugomi atviru tekstu.
- 2012 ne pelno grupė *D33D* iš *Yahoo!* pavogė ir viešai paskelbė 450,000 vartotojų prisijungimo duomenų. Slaptažodžiai buvo saugomi atviru tekstu.